



Fraktal renderovanje sa hibridnom paralelizacijom

Diplomski rad

Tamaš Pap

Mentor: dr Veljko Petrović

Fakultet tehničkih nauka, Novi Sad
Računarstvo i automatika

2026

Šta su fraktali i čemu služe?

- ▶ Oblik čija se složenost **ne gubi uvećanjem** i koji je samosličan, sa fraktalnom dimenzijom koja nije ceo broj
- ▶ U prirodi: obalske linije, rečne mreže, oblaci, otkucaji srca, pluća i krvni sudovi (grananje)

Čemu služe:

- ▶ kompresija slike (iterirani sistemi funkcija)
- ▶ kompaktne fraktalne antene za rad na više frekvencija
- ▶ proceduralna grafika (teren, oblaci, teksture)
- ▶ dijagnostika u medicini: fraktalna dimenzija tkiva i krvnih sudova razlikuje zdravo od obolelog



Kohova kriva



Sjerpinski trougao



Vicekov fraktal



Sjerpinski tepih

Motivacija i cilj rada

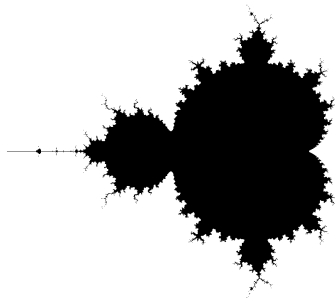
- ▶ Mandelbrotov skup: jednostavno pravilo daje granicu beskonačne detaljnosti; *escape-time* algoritam je potpuno paralelan po pikselu
- ▶ Većina renderera koristi samo jedno CPU jezgro **ili** samo GPU
- ▶ Tipičan računar već ima višejezgarni CPU **i** GPU

Cilj: renderer koji gura CPU i GPU do granice, pa ih spaja raspoređivačem

Mandelbrotov skup i *escape-time* algoritam

$$z_0 = 0, \quad z_{n+1} = z_n^2 + c$$
$$M = \{ c \in \mathbb{C} : (z_n) \text{ ograničen} \}$$

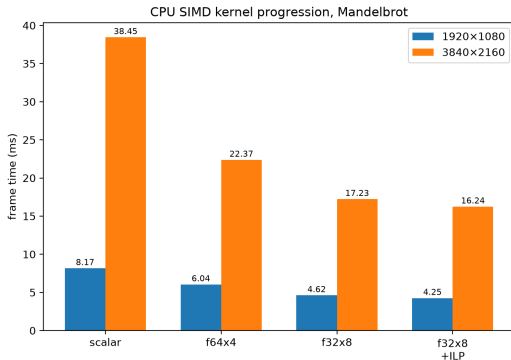
- ▶ ograničen ($|c| \leq 2$), povezan, samosličan
- ▶ test: prag bekstva $|z_n| > 2 + \text{max_iter}$
- ▶ broj iteracija je jako neuravnotežen po slici, što je glavni izazov za paralelizaciju



CPU renderer: skalarna varijanta i SIMD

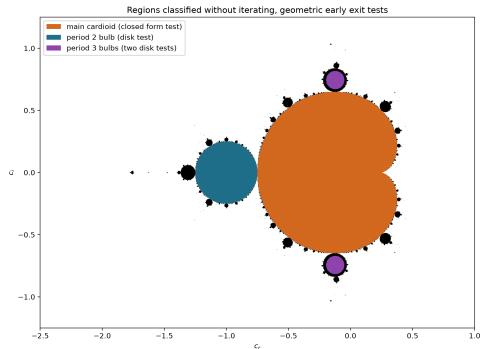
- ▶ $f64$ + FMA, test bekstva kao $|z_n|^2 > 4$
- ▶ Brentova detekcija ciklusa prepoznaje periodične piksele bez čekanja na `max_iter`
- ▶ SIMD: $f32 \times 8$ / $f64 \times 4$ + maska aktivnih traka; dva nezavisna lanca za ILP

do $2.4\times$ ubrzanje nad skalarnom osnovom



Rani izlazak, Mariani-Silver, procena rastojanja

- ▶ Kardioida i period-2 balon: tačan zatvoren oblik, bez iteracije
- ▶ Mariani-Silver: testira se samo ivica pravougaonika; ako je uniformna, popunjava se bez računanja unutrašnjosti
- ▶ Procena rastojanja (DEM): meri koliko je piksel daleko od granice, koristi se kao dodatni test odsecanja



Duboki zum i neočekivani rezultati

- ▶ **Teorija perturbacije** ($\delta_n = z_n - Z_n$, rebasing, do 8 referenci) + **double-double** referentna orbita za zum $> 10^{12}$
- ▶ Dve optimizacije koje **nisu** isplatile na ovom hardveru:
 - perturbacija ne donosi dobitak do zuma 10^{12} (**f64** je dovoljan)
 - Hilbertova kriva (CPU obilazak) je *sporija* od reda po redovima

GPU implementacija: WebGPU i CUDA

- ▶ **WebGPU** (`wgpu`) kompajlira se u Vulkan, Metal ili DirectX preko drajvera; WGSL nema `f64`, pa je kernel isključivo `f32`
- ▶ Isti WGSL kernel deli sve tipove fraktala; dve ulazne tačke: ceo kadar (radne grupe 16×16) i pojedinačne pločice za hibridni raspoređivač
- ▶ **CUDA** cilja `sm_86` (Ampere); PTX se kompajlira unapred pomoću `nvcc`, bez kompajliranja u realnom vremenu
- ▶ CUDA zadržava pun `f64` paritet sa CPU-om; `--ftz=false` čuva denormalizovane brojeve, potrebne za tačnost glatkog bojenja blizu praga bekstva

	WebGPU	CUDA
prenosivost	Vulkan / Metal / DirectX	samo NVIDIA
preciznost	samo <code>f32</code>	<code>f32</code> i <code>f64</code>
kompajliranje	WGSL, u realnom vremenu	PTX, unapred (<code>nvcc</code>)
kadar (1080p, zum 1)	2.26 ms	1.61 ms

GPU optimizacije

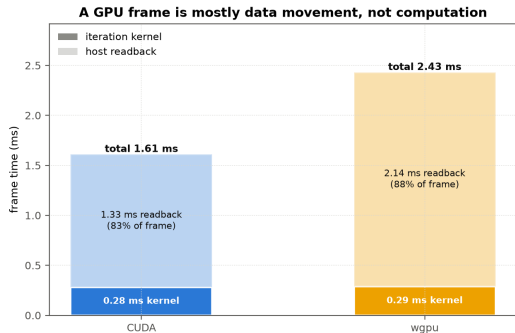
- ▶ `--fmad=false` isključuje sažimanje množenja i sabiranja, pa CUDA zaokružuje bit-identično sa CPU kernelom
- ▶ `colorize()` premešten na GPU (histogram + očitavanje iz palete kao poseban kernel), tačno do piksela u odnosu na CPU verziju
- ▶ Mortonov (Z) redosled niti, isproban za bolju lokalnost unutar *warp*-a, donosi samo +1,9%, u granicama šuma

colorize na GPU-u: 54–63% brže od početka do kraja

pri plitkim i srednjim uvećanjima, gde je bojenje ranije dominiralo kadrom

Usko grlo: PCIe magistrala

- ▶ Diskretni GPU je dostupan samo preko **PCIe 3.0 x8** (~ 7.9 GB/s teorijski)
- ▶ Integrirani GPU deli sistemsku RAM sa CPU-om, bez PCIe prenosa
- ▶ Kadar $1920 \times 1080 = 8.29$ MB koji se prenosi nazad na *svaki* kadar
- ▶ *Pinned memory* omogućava DMA direktno u bafer pozivaoca



80–90% vremena GPU kadra je prenos

Hibridni CPU+GPU raspoređivač

Zašto:

- ▶ GPU niti rade u *warp*-ovima od 32, u lok-koraku, pa jedan spor piksel usporava ceo *warp*
- ▶ CPU nit je nezavisna, pa su divergentne granične oblasti jeftinije na CPU-u

Dizajn (4 koraka):

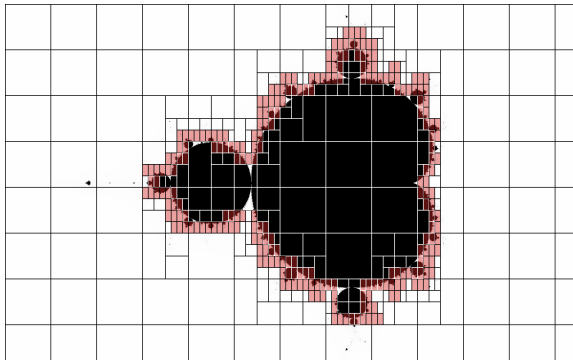
1. klasifikacija pločica (4 temena)
2. raspodela: GPU odmah, CPU iz reda
3. krađa posla unutar kadra
4. PI regulator između kadrova

Klasifikacija pločica i PI regulator

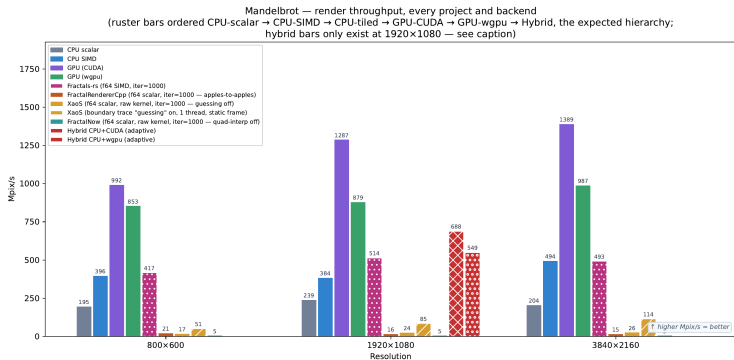
$$s = \frac{\max(i_k) - \min(i_k)}{N}$$

$$\text{threshold} \leftarrow K_P \text{err} + K_I \overline{\text{err}}$$

- ▶ $s < \text{prag}$: pločica ide na GPU, inače rekurzivna podela
- ▶ $\text{err} = t_{\text{gpu}} - t_{\text{cpu}}$, klizni prosek od 16 kadrova



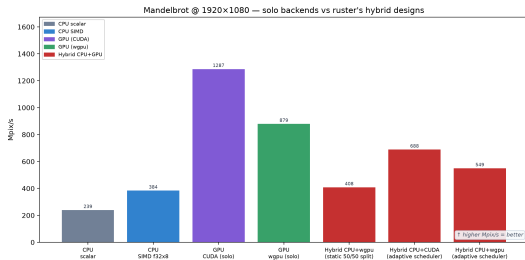
Rezultati: propusnost renderovanja



CUDA: 1389 Mpix/s na 4K

AMD Ryzen 7 5800H + RTX 3050 Laptop, Mandelbrotov skup, max_iter=1000

Rezultati: raspoređivač pri plitkom zumu

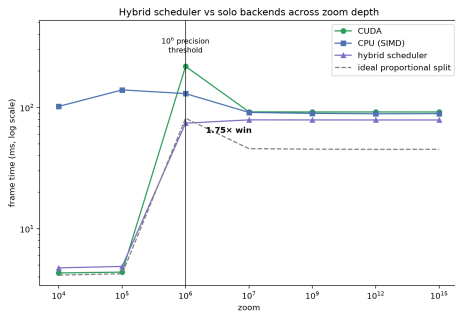


- Pri plitkom zumu, samostalni GPU je već dovoljno brz i skoro uvek pobeđuje
- Raspoređivač tu samo dodaje trošak klasifikacije, bez koristi

gubi od samostalnog GPU-a

nema neravnoteže između uređaja koju bi ispravio

Rezultati: raspoređivač pri dubokom zumu



konfiguracija	vreme
CPU (solo)	130 ms
"idealna" podela	81.6 ms
hibrid	74.3 ms

1.75× brže od CPU-a

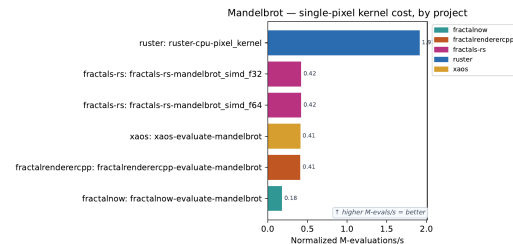
1.10× brže od "idealne" podele

Na pragu preciznosti 10^6 GPU prelazi na sporiju f64

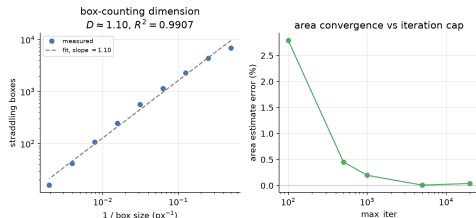
aritmetiku, pa CPU postaje brži solo uređaj

"Idealna" podela samo deli posao proporcionalno izmerenoj brzini uređaja, pod pretpostavkom da je svaki piksel podjednako skup. Stvarna granica skupa mnogo je skuplja od unutrašnjosti; klasifikator pločica to prepoznaje i usmerava skupe delove ka boljem uređaju, pa hibrid nadmašuje čak i tu "idealnu" podelu.

Poređenje sa drugim rendererima i validacija



Trošak samog kernela po pikselu (miliona evaluacija/s, normalizovano), bez niti, bojenja i "nagađanja" kod XaoS-a i FractalNow-a: ruster je red veličine brži od najbržeg konkurenta, dva reda od najsporijeg



Box-counting dimenzija ≈ 1.10 ; konvergencija površine ka 1.50659177

Zaključak

- ▶ **CPU strana važnija nego očekivano:** rani izlazak i detekcija ciklusa daju $\sim 16\times$ nad naivnim C++ rendererom, bez GPU-a
- ▶ **GPU dominira transferom i bojenjem, ne kernelom:** premeštanje bojenja na uređaj je najveći pojedinačni dobitak
- ▶ **Raspoređivač se isplati tačno na pragu preciznosti,** gubi kad su uređaji neuravnoteženi

Budući rad: `colorize()` na GPU za wgpu i hibrid; podešavanje raspoređivača na dubokom zumu; perturbacija na stvarno dubokim zumovima; poređenje sa novim GPU-konkurentnim rešenjem koje je nedavno objavio jedan poznati render fraktala.

Hvala na pažnji



Pitanja?